

Unicode in the Library, Part 1: UTF Transcoding

Document #: P2728R0
Date: 2022-11-20
Project: Programming Language C++
Audience: SG-16 Unicode
LEWG-I
LEWG
Reply-to: Zach Laine
<whatwasthataddress@gmail.com>

Contents

- 1 Motivation
 - 1.1 A note about P1629
- 2 The shortest Unicode primer imaginable
- 3 Use cases
 - 3.1 Case 1: Transcode a buffer as fast as possible
 - 3.2 Case 2: Transcode an object as fast as possible
 - 3.3 Case 3: Transcode an object as conveniently as possible
 - 3.4 Case 4: Adapt to an existing iterator interface taking a different UTF
 - 3.5 Case 5: Adapt to an existing range interface taking a different UTF
- 4 Proposed design
 - 4.1 Dependencies
 - 4.2 Add concepts that describe parameters to transcoding APIs
 - 4.3 Add a standard null-terminated sequence sentinel
 - 4.4 Add constants and utility functions that query the state of UTF sequences (well-formedness, etc.)
 - 4.5 Add transcode algorithms
 - 4.6 Add the transcoding iterators
 - 4.6.1 First, the basic ones
 - 4.6.2 Add out and insert transcoding iterators

- 4.6.3 Add factory functions for all the transcoding iterators
- 4.7 Add transcoding views
 - 4.7.1 Add the views proper
 - 4.7.2 Add `as_utfN()`
 - 4.7.3 Add `utfN_view` specializations of `formatter`
- 4.8 Add a feature test macro
- 4.9 Design notes
 - 4.9.1 Iterator “unpacking”
- 5 Implementation experience
- 6 Performance
 - 6.1 UTF-8 to UTF-16
 - 6.2 UTF-8 to UTF-32
- 7 References

§ 1 Motivation

Unicode is important to many, many users in everyday software. It is not exotic or weird. Well, it’s weird, but it’s not weird to see it used. C and C++ are the only major production languages with essentially no support for Unicode.

Let’s fix.

To fix, first we start with the most basic representations of strings in Unicode: UTF. You might get a UTF string from anywhere; on Windows you often get them from the OS, in UTF-16. In web-adjacent applications, strings are most commonly in UTF-8. In ASCII-only applications, everything is in UTF-8, by its definition as a superset of ASCII.

Often, an application needs to switch between UTFs: 8 -> 16, 32 -> 16, etc. In SG-16 we’ve taken to calling such UTF-N -> UTF-M operations “transcoding”.

I’m proposing interfaces to do transcoding that meet certain design requirements that I think are important; I hope you’ll agree:

- Ranges are the future. We should have range-friendly ways of doing transcoding. This includes support for sentinels and lazy views.
- Iterators are the present. We should support generic programming, whether it is done in terms of pointers, a particular iterator, or an iterator type specified as a template parameter.
- Pointers are important. The fastest transcoding will always be done in terms of pointers (due to the speed of SIMD operations in particular). We should support all-

pointer-interface transcoding.

- Transcoding cannot be a black box; sometimes you need to be able to find where there is a break in the encoding, or to detect whether a sequence has any broken encodings in it. We should provide utility functions that let users investigate these states.
- A null-terminated string should not be treated as a special case. The ubiquity of such strings means that they should be treated as first-class strings.
- It is common to want to view the same text as code points and code units at different times. It is therefore important that transcoding iterators have a convenient way to access the underlying sequence of code units being transcribed.

§ 1.1 A note about P1629

[P1629R1] from JeanHeyd Meneide is a much more ambitious proposal that aims to standardize a general-purpose text encoding conversion mechanism. This proposal is not at odds with P1629; the two proposals have largely orthogonal aims. This proposal only concerns itself with UTF interconversions, which is all that is required for Unicode support. P1629 is concerned with those conversions, plus a lot more. Accepting both proposals would not cause problems; in fact, the APIs proposed here could be used to implement parts of the P1629 design.

§ 2 The shortest Unicode primer imaginable

There are multiple encoding types defined in Unicode: UTF-8, UTF-16, and UTF-32.

A *code unit* is the lowest-level datum-type in your Unicode data. Examples are a `char` in UTF-8 and a `uint32_t` in UTF-32.

A *code point* is a 32-bit integral value that represents a single Unicode value. Examples are U+0041 “A” “LATIN CAPITAL LETTER A” and U+0308 “” “COMBINING DIAERESIS”.

A code point may be consist of multiple code units. For instance, 3 UTF-8 code units in sequence may encode a particular code unit.

§ 3 Use cases

There are several contexts to consider here:

- Coding convenience, terseness, and clarity.
- Performance.
- Compatibility with other APIs.

Some times the contexts are in alignment, and sometimes they are in conflict. We need multiple ways to do transcoding to cover all of those contexts. Let's make things more concrete:

§ 3.1 Case 1: Transcode a buffer as fast as possible

We care primarily about performance in this use case, so everything is a pointer. Also, our wire-communications layer knows nothing about the UTFs, so we need to use some of the utility functions to make sure we don't process partially-received UTF-8 sequences.

```
// Using same size to ensure the transcode operation always has room.
char utf8_buf[buf_size];
char utf16_buf[buf_size];

char * read_first = utf8_buf;
while (true) {
    // Reads off a wire; may contain partial UTF-8 sequences at the ends
    of
    // some reads.
    char * buf_last = read_into_utf8_buffer(read_first, utf8_buf +
        buf_size);

    if (buf_last == read_first)
        continue;

    // find the last whole UTF-8 sequence, so we don't feed partial
    sequences
    // to the algorithm below.
    char * last = buf_last;
    auto const last_lead = std::ranges::find_last_if(
        utf8_buf, buf_last, std::uc::lead_code_unit);
    if (!last_lead.empty()) {
        auto const dist_from_end = buf_last - last_lead.begin();
        assert(dist_from_end <= 4);
        if (std::uc::utf8_code_units(*last_lead.begin()) != dist_from_end)
            last = last_lead.begin();
    }

    // Same interface as std::ranges::copy(), except that it converts as
    it copies.
    auto const result = std::uc::transcode_to_utf16(utf8_buf, last,
        utf16_buf);

    // Do something with the resulting UTF-16 buffer contents.
    send_utf16_somewhere(utf16_buf, result.out);

    // Copy partial UTF-8 sequence to start of buffer.
    read_first = std::ranges::copy_backward(last, buf_last, utf8_buf).out;
}
```

§ 3.2 Case 2: Transcode an object as fast as possible

my_string does not provide a pointer-based interface, so we need to use more generic facilities for this case.

```
struct my_string; // Some string type with *non-pointer* iterators.

my_string input = get_utf8_input();
std::vector<uint16_t> input_as_utf16(input.size()); // Reserve some space.
auto const result = std::uc::transcode_to_utf16(input,
    input_as_utf16.data());
input_as_utf16.resize(result.out - input_as_utf16.data()); // Trim unused space.
```

§ 3.3 Case 3: Transcode an object as conveniently as possible

This solution is similar to Case 2, but marginally more convenient. There are other cases, like accepting output from std algorithms, that indicate use of a back-inserter.

```
struct my_string; // Some string type with *non-pointer* iterators.

my_string input = get_utf8_input();
std::vector<uint16_t> input_as_utf16;
std::ranges::copy(input,
    std::uc::from_utf8_back_inserter(input_as_utf16));
```

§ 3.4 Case 4: Adapt to an existing iterator interface taking a different UTF

This time, we have a generic iterator interface we want to transcode into, so we want to use the transcoding iterators.

```
// A generic function that accepts sequences of UTF-16.
template<class UTF16Iter>
void process_input(UTF16Iter first, UTF16Iter last);

std::string input = get_utf8_input(); // A std::string used as a UTF-8 string.

process_input(std::uc::utf_8_to_16_iterator(input.begin(), input.begin(),
    input.end()),
    std::uc::utf_8_to_16_iterator(input.begin(), input.end(),
    input.end()));

// Even more conveniently:
auto const utf16_view = std::uc::as_utf16(input);
process_input(utf16_view.begin(), utf16_view.end());
```

§ 3.5 Case 5: Adapt to an existing range interface taking a different UTF

In this case, we have a generic range interface to transcode into, so we use a transcoding view.

```

// A generic function that accepts sequences of UTF-16.
template<class UTF16Range>
void process_input(UTF16Range && r);

std::string input = get_utf8_input(); // A std::string used as a UTF-8
string.

process_input(std::uc::as_utf16(input));

```

§ 4 Proposed design

§ 4.1 Dependencies

This proposal depends on the existence of [P2727](#) “std::iterator_interface”.

§ 4.2 Add concepts that describe parameters to transcoding APIs

```

namespace std::uc {

    enum class format { utf8 = 1, utf16 = 2, utf32 = 4 };

    template<class T, format F>
    concept code_unit = integral<T> && sizeof(T) == (int)F;

    template<class T>
    concept utf8_code_unit = code_unit<T, format::utf8>;

    template<class T>
    concept utf16_code_unit = code_unit<T, format::utf16>;

    template<class T>
    concept utf32_code_unit = code_unit<T, format::utf32>;

    template<class T>
    concept utf_code_unit = utf8_code_unit<T> || utf16_code_unit<T> ||
        utf32_code_unit<T>;

    template<class T, format F>
    concept code_unit_iter =
        bidirectional_iterator<T> && code_unit<iter_value_t<T>, F>;
    template<class T, format F>
    concept code_unit_pointer =
        is_pointer_v<T> && code_unit<iter_value_t<T>, F>;
    template<class T, format F>
    concept code_unit_range = ranges::bidirectional_range<T> &&
        code_unit<ranges::range_value_t<T>, F>;

    template<class T>
    concept utf8_iter = code_unit_iter<T, format::utf8>;
    template<class T>
    concept utf8_pointer = code_unit_pointer<T, format::utf8>;
    template<class T>

```

```

    concept utf8_range = code_unit_range<T, format::utf8>;

template<class T>
    concept utf16_iter = code_unit_iter<T, format::utf16>;
template<class T>
    concept utf16_pointer = code_unit_pointer<T, format::utf16>;
template<class T>
    concept utf16_range = code_unit_range<T, format::utf16>;

template<class T>
    concept utf32_iter = code_unit_iter<T, format::utf32>;
template<class T>
    concept utf32_pointer = code_unit_pointer<T, format::utf32>;
template<class T>
    concept utf32_range = code_unit_range<T, format::utf32>;

template<class T>
    concept utf_iter = utf8_iter<T> || utf16_iter<T> || utf32_iter<T>;
template<class T>
    concept utf_pointer = utf8_pointer<T> || utf16_pointer<T> ||
        utf32_pointer<T>;
template<class T>
    concept utf_range = utf8_range<T> || utf16_range<T> || utf32_range<T>;

template<class T>
    concept utf_range_like =
        utf_range<remove_reference_t<T>> ||
        utf_pointer<remove_reference_t<T>>;

template<class T>
    concept utf8_input_range_like =
        (ranges::input_range<remove_reference_t<T>> &&
         utf8_code_unit<iter_value_t<T>>) ||
        utf8_pointer<remove_reference_t<T>>;
template<class T>
    concept utf16_input_range_like =
        (ranges::input_range<remove_reference_t<T>> &&
         utf16_code_unit<iter_value_t<T>>) ||
        utf16_pointer<remove_reference_t<T>>;
template<class T>
    concept utf32_input_range_like =
        (ranges::input_range<remove_reference_t<T>> &&
         utf32_code_unit<iter_value_t<T>>) ||
        utf32_pointer<remove_reference_t<T>>;

template<class T>
    concept utf_input_range_like =
        utf8_input_range_like<T> || utf16_input_range_like<T> ||
        utf32_input_range_like<T>;
}

```

§ 4.3 Add a standard null-terminated sequence sentinel

```

namespace std::uc {
    struct null_sentinel_t {

```

```

constexpr null_sentinel_t base() const { return {}; }

template<class T>
    requires utf8_code_unit<T> || utf16_code_unit<T> ||
             utf32_code_unit<T>
    friend constexpr bool operator==(T* p, null_sentinel_t);
};

inline constexpr null_sentinel_t null_sentinel;
}

```

The `base()` member bears explanation. It is there to make iterator/sentinel pairs easy to use in a generic context. Consider a range `r1` of code points delimited by a pair of `utf_8_to_32_iterator<char const*>` transcoding iterators (defined later in this paper). The range of underlying UTF-8 code units is `[r1.begin().base(), r1.end().base())`.

Now consider a range `r2` of code points that is delimited by a `utf_8_to_32_iterator<char const*>` transcoding iterator and a `null_sentinel`. Now our underlying range of UTF-8 is `[r.begin().base(), null_sentinel)`.

Instead of making people writing generic code have to special-case the use of `null_sentinel`, `null_sentinel` has a `base()` member that lets us write `r.end().base()` instead of `null_sentinel`. This means that for either `r` or `r2`, the underlying range of UTF-8 code units is just `[r1.begin().base(), r1.end().base())`.

§ 4.4 Add constants and utility functions that query the state of UTF sequences (well-formedness, etc.)

```

namespace std::uc {
    inline constexpr uint16_t high_surrogate_base = 0xd7c0;
    inline constexpr uint16_t low_surrogate_base = 0xdc00;
    inline constexpr uint32_t high_surrogate_min = 0xd800;
    inline constexpr uint32_t high_surrogate_max = 0xdbff;
    inline constexpr uint32_t low_surrogate_min = 0xdc00;
    inline constexpr uint32_t low_surrogate_max = 0xdfff;
    inline constexpr uint32_t replacement_character = 0xfffd;

    // Returns is_high_surrogate(c) || is_low_surrogate(c).
    constexpr bool is_surrogate(uint32_t c);

    // Returns true iff c is a Unicode high surrogate.
    constexpr bool is_high_surrogate(uint32_t c);

    // Returns true iff c is a Unicode low surrogate.
    constexpr bool is_low_surrogate(uint32_t c);

    // Returns true iff c is a Unicode reserved noncharacter.
    constexpr bool is_reserved_noncharacter(uint32_t c);

    // Returns true iff c is a valid Unicode scalar value.
    constexpr bool is_scalar_value(uint32_t c);
}

```



```

// Returns true iff c is a Unicode scalar value not in the reserved
// range.
constexpr bool is_unreserved_scalar_value(uint32_t c);

// Returns true iff c is a UTF-8 lead code unit (which must be followed
// by 1-3 following units).
constexpr bool is_lead_code_unit(unsigned char c);

// Returns true iff c is a UTF-8 continuation (non-lead) code unit.
constexpr bool is_continuation(unsigned char c);

// Given the first (and possibly only) code unit of a UTF-8-encoded code
// point, returns the number of bytes occupied by that code point (in
the
// range [1, 4]). Returns a value < 0 if first_unit is not a valid
// initial UTF-8 code unit.
constexpr int utf8_code_units(unsigned char first_unit);

// Given the first (and possibly only) code unit of a UTF-16-encoded
code
// point, returns the number of code units occupied by that code point
// (in the range [1, 2]). Returns a value < 0 if first_unit is
// not a valid initial UTF-16 code unit.
constexpr int utf16_code_units(uint16_t first_unit);

// Returns the first code unit in [first, last) that is not properly
// UTF-8 encoded, or last if no such code unit is found.
template<utf8_iter I, sentinel_for<I> S = I>
    requires random_access_iterator<I>
    constexpr I find_invalid_encoding(I first, S last);

// Returns the first code unit in [first, last) that is not properly
// UTF-16 encoded, or last if no such code unit is found.
template<utf16_iter I, sentinel_for<I> S = I>
    requires random_access_iterator<I>
    constexpr I find_invalid_encoding(I first, S last);

// Returns true iff [first, last) is properly UTF-8 encoded.
template<utf8_iter I>
    requires random_access_iterator<I>
    constexpr bool is_encoded(I first, I last);

// Returns true iff [first, last) is properly UTF-16 encoded
template<utf16_iter I>
    requires random_access_iterator<I>
    constexpr bool is_encoded(I first, I last);

// Returns true iff [first, last) is empty or the initial UTF-8 code
// units in [first, last) form a valid Unicode code point.
template<utf8_iter I>
    requires random_access_iterator<I>
    constexpr bool starts_encoded(I first, I last);

// Returns true iff [first, last) is empty or the initial UTF-16 code
// units in [first, last) form a valid Unicode code point.

```

```

template<utf16_iter I>
    requires random_access_iterator<I>
    constexpr bool starts_encoded(I first, I last);

// Returns true iff [first, last) is empty or the final UTF-8 code
// units in [first, last) form a valid Unicode code point.
template<utf8_iter I>
    requires random_access_iterator<I>
    constexpr bool ends_encoded(I first, I last);

// Returns true iff [first, last) is empty or the final UTF-16 code
// units in [first, last) form a valid Unicode code point.
template<utf16_iter I>
    requires random_access_iterator<I>
    constexpr bool ends_encoded(I first, I last);
}

```

§ 4.5 Add transcode algorithms

These algorithms take an iterator/sentinel pair, a range, or a null-terminated string, and transcode the input to the output iterator.

```

namespace std::uc {
    // An alias for in_out_result returned by algorithms that perform a
    // transcoding copy.
    template<class Iter, class OutIter>
    using transcode_result = in_out_result<Iter, OutIter>;

    template<class T>
    using range-like-result-iterator = see below; // exposition only

    // -> UTF-8

    template<input_iterator I, sentinel_for<I> S, output_iterator<char> O>
        requires(utf16_code_unit<iter_value_t<I>> ||
                 utf32_code_unit<iter_value_t<I>>)
    constexpr transcode_result<I, O> transcode_to_utf8(I first, S last,
        O out);

    template<class R, output_iterator<uint32_t> O>
        requires(utf16_input_range_like<R> || utf32_input_range_like<R>)
    constexpr transcode_result<range-like-result-iterator<R>, O>
        transcode_to_utf8(R&& r, O out);

    // -> UTF-16

    template<input_iterator I, sentinel_for<I> S, output_iterator<char> O>
        requires(utf8_code_unit<iter_value_t<I>> ||
                 utf32_code_unit<iter_value_t<I>>)
    constexpr transcode_result<I, O> transcode_to_utf16(I first, S last,
        O out);

    template<class R, output_iterator<uint32_t> O>
        requires(utf8_input_range_like<R> || utf32_input_range_like<R>)

```

```

        constexpr transcode_result<range-like-result-iterator<R>, 0>
        transcode_to_utf16(R&& r, 0 out);

// -> UTF-32

template<input_iterator I, sentinel_for<I> S, output_iterator<char> O>
        requires(utf8_code_unit<iter_value_t<I>> ||
        utf16_code_unit<iter_value_t<I>>))
constexpr transcode_result<I, 0> transcode_to_utf32(I first, S last,
        0 out);

template<class R, output_iterator<uint32_t> O>
        requires(utf8_input_range_like<R> || utf16_input_range_like<R>)
        constexpr transcode_result<range-like-result-iterator<R>, 0>
        transcode_to_utf32(R&& r, 0 out);
}

```

`range-like-result-iterator<T>` is `T` if `is_pointer_v<remove_reference_t<T>>` is true, and `ranges::borrowed_iterator_t<T>` otherwise.

§ 4.6 Add the transcoding iterators

§ 4.6.1 First, the basic ones

I'm using [P2727](#)'s `iterator_interface` here for simplicity.

```

namespace std::uc {
    // An error handler type that can be used with the converting iterators;
    // provides the Unicode replacement character on errors.
    struct use_replacement_character {
        constexpr uint32_t operator()(const char*) const { return
        replacement_character; }
    };

    template<
        utf32_iter I,
        sentinel_for<I> S = I,
        transcoding_error_handler ErrorHandler = use_replacement_character>
    struct utf_32_to_8_iterator
        : iterator_interface<utf_32_to_8_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag, char, char> {
        constexpr utf_32_to_8_iterator();
        explicit constexpr utf_32_to_8_iterator(I first, I it, S last);
        template<class I2, class S2>
            requires convertible_to<I2, I> && convertible_to<S2, S>
            constexpr utf_32_to_8_iterator(
                const utf_32_to_8_iterator<I2, S2, ErrorHandler>& other);

        constexpr I begin() const { return first_; }
        constexpr S end() const { return last_; }

        constexpr char operator*() const { return buf_[index_]; }

        constexpr I base() const { return it_; }
    }
}

```

```

constexpr utf_32_to_8_iterator& operator++();
constexpr utf_32_to_8_iterator& operator--();

template<
    class I1, class S1,
    class I2, class S2,
    class ErrorHandler2>
friend constexpr bool operator==(
    const utf_32_to_8_iterator<I1, S1, ErrorHandler2>& lhs,
    const utf_32_to_8_iterator<I2, S2, ErrorHandler2>& rhs)
    requires requires { lhs.base() == rhs.base(); }
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    friend constexpr bool operator==(utf_32_to_8_iterator lhs,
    utf_32_to_8_iterator rhs)
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

using base-type =          // exposition only
    iterator_interface<utf_32_to_8_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag,
        uint32_t,
        uint32_t>;
using base-type::operator++;
using base-type::operator--;

private:
    I first_;                // exposition only
    I it_;                   // exposition only
    S last_;                 // exposition only
    int index_;              // exposition only
    array<char, 5> buf_;     // exposition only

    template<
        utf32_iter I2,
        sentinel_for<I2> S2,
        transcoding_error_handler ErrorHandler2>
    friend struct utf_32_to_8_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    utf_32_to_8_iterator<I, S, ErrorHandler> lhs, S rhs)
    requires requires { lhs.base() == rhs; };

template<
    utf8_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_8_to_32_iterator
    : iterator_interface<utf_8_to_32_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag, uint32_t, uint32_t> {
    constexpr utf_8_to_32_iterator();
    explicit constexpr utf_8_to_32_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>

```

```

        constexpr utf_8_to_32_iterator(
            const utf_8_to_32_iterator<I2, S2, ErrorHandler>& other);

constexpr I begin() const { return first_; }
constexpr S end() const { return last_; }

constexpr uint32_t operator*() const;

constexpr I base() const { return it_; }

constexpr utf_8_to_32_iterator& operator++();
constexpr utf_8_to_32_iterator& operator--();

        friend constexpr bool operator==(utf_8_to_32_iterator lhs,
            utf_8_to_32_iterator rhs)
        { return lhs.base() == rhs.base(); }

using base-type =          // exposition only
    iterator_interface<utf_8_to_32_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag,
        uint32_t,
        uint32_t>;
using base-type::operator++;
using base-type::operator--;

private:
    I first_;                // exposition only
    I it_;                   // exposition only
    S last_;                 // exposition only

    template<
        utf8_iter I2,
        sentinel_for<I2> S2,
        transcoding_error_handler ErrorHandler2>
    friend struct utf_8_to_16_iterator;

    template<
        utf8_iter I2,
        sentinel_for<I2> S2,
        transcoding_error_handler ErrorHandler2>
    friend struct utf_8_to_32_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_8_to_32_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<
    class I1, class S1,
    class I2, class S2,
    class ErrorHandler>
constexpr bool operator==(
    const utf_8_to_32_iterator<I1, S1, ErrorHandler>& lhs,
    const utf_8_to_32_iterator<I2, S2, ErrorHandler>& rhs)
    requires requires { lhs.base() == rhs.base(); };

```

```

template<
    utf32_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_32_to_16_iterator
    : iterator_interface<utf_32_to_16_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag, uint16_t, uint16_t> {
    constexpr utf_32_to_16_iterator();
    explicit constexpr utf_32_to_16_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_32_to_16_iterator(
            const utf_32_to_16_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr uint16_t operator*() const
    { return buf_[index_]; }

    constexpr I base() const { return it_; }

    constexpr utf_32_to_16_iterator& operator++();
    constexpr utf_32_to_16_iterator& operator--();

    template<
        class I1, class S1,
        class I2, class S2,
        class ErrorHandler2>
    friend constexpr bool operator==(
        const utf_32_to_16_iterator<I1, S1, ErrorHandler2>& lhs,
        const utf_32_to_16_iterator<I2, S2, ErrorHandler2>& rhs)
        requires requires { lhs.base() == rhs.base(); }
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

        friend constexpr bool operator==(utf_32_to_16_iterator lhs,
            utf_32_to_16_iterator rhs)
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    using base-type =          // exposition only
        iterator_interface<utf_32_to_16_iterator<I, S, ErrorHandler>,
            bidirectional_iterator_tag,
            uint16_t,
            uint16_t>;
    using base-type::operator++;
    using base-type::operator--;

private:
    I first_;                // exposition only
    I it_;                   // exposition only
    S last_;                 // exposition only
    int index_;              // exposition only
    array<uint16_t, 4> buf_;  // exposition only

    template<

```

```

        utf32_iter I2,
        sentinel_for<I2> S2,
        transcoding_error_handler ErrorHandler2>
friend struct utf_32_to_16_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_32_to_16_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<
    utf16_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_16_to_32_iterator
    : iterator_interface<utf_16_to_32_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag, uint32_t, uint32_t> {
    constexpr utf_16_to_32_iterator();
    explicit constexpr utf_16_to_32_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_16_to_32_iterator(
            const utf_16_to_32_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr uint32_t operator*() const;

    constexpr I base() const { return it_; }

    constexpr utf_16_to_32_iterator& operator++();
    constexpr utf_16_to_32_iterator& operator--();

    friend constexpr bool operator==(utf_16_to_32_iterator lhs,
        utf_16_to_32_iterator rhs)
    { return lhs.base() == rhs.base(); }

    using base-type =          // exposition only
        iterator_interface<utf_16_to_32_iterator<I, S, ErrorHandler>,
            bidirectional_iterator_tag,
            uint32_t,
            uint32_t>;
    using base-type::operator++;
    using base-type::operator--;

private:
    I first_;                // exposition only
    I it_;                   // exposition only
    S last_;                 // exposition only

    template<
        utf32_iter I2,
        sentinel_for<I2> S2,
        transcoding_error_handler ErrorHandler2>

```

```

friend struct utf_32_to_16_iterator;

template<
    utf16_iter I2,
    sentinel_for<I2> S2,
    transcoding_error_handler ErrorHandler2>
friend struct utf_16_to_32_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_32_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<
    class I1, class S1,
    class I2, class S2,
    class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_32_iterator<I1, S1, ErrorHandler>& lhs,
    const utf_16_to_32_iterator<I2, S2, ErrorHandler>& rhs)
    requires requires { lhs.base() == rhs.base(); };

template<
    utf16_iter I,
    sentinel_for<I> S = I,
    transcoding_error_handler ErrorHandler = use_replacement_character>
struct utf_16_to_8_iterator
    : iterator_interface<utf_16_to_8_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag, char, char> {
    constexpr utf_16_to_8_iterator();
    explicit constexpr utf_16_to_8_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_16_to_8_iterator(const utf_16_to_8_iterator<I2, S2>&
            other);

    constexpr I begin() const { return first_; }
    constexpr S end() const { return last_; }

    constexpr char operator*() const { return buf_[index_]; }

    constexpr I base() const { return it_; }

    constexpr utf_16_to_8_iterator& operator++();
    constexpr utf_16_to_8_iterator& operator--();

    template<
        class I1, class S1,
        class I2, class S2,
        class ErrorHandler2>
    friend constexpr bool operator==(
        const utf_16_to_8_iterator<I1, S1, ErrorHandler2>& lhs,
        const utf_16_to_8_iterator<I2, S2, ErrorHandler2>& rhs)
        requires requires { lhs.base() == rhs.base(); }
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

```



```

        friend constexpr bool operator==(utf_16_to_8_iterator lhs,
        utf_16_to_8_iterator rhs)
        { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

using base-type =          // exposition only
    iterator_interface<utf_16_to_8_iterator<I, S, ErrorHandler>,
                        bidirectional_iterator_tag,
                        char,
                        char>;
using base-type::operator++;
using base-type::operator--;

private:
    I first_;                // exposition only
    I it_;                   // exposition only
    S last_;                 // exposition only
    int index_;              // exposition only
    array<char, 5> buf_;     // exposition only

    template<
        utf16_iter I2,
        sentinel_for<I2> S2,
        transcoding_error_handler ErrorHandler2>
    friend struct utf_16_to_8_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_8_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };

template<
    class I1, class S1,
    class I2, class S2,
    class ErrorHandler>
constexpr bool operator==(
    const utf_16_to_8_iterator<I1, S1, ErrorHandler>& lhs,
    const utf_16_to_8_iterator<I2, S2, ErrorHandler>& rhs)
    requires requires { lhs.base() == rhs.base(); };

template<
    utf8_iter I,
    sentinel_for<I> S,
    transcoding_error_handler ErrorHandler>
struct utf_8_to_16_iterator
    : iterator_interface<utf_8_to_16_iterator<I, S, ErrorHandler>,
                        bidirectional_iterator_tag, uint16_t, uint16_t> {
    constexpr utf_8_to_16_iterator();
    explicit constexpr utf_8_to_16_iterator(I first, I it, S last);
    template<class I2, class S2>
        requires convertible_to<I2, I> && convertible_to<S2, S>
        constexpr utf_8_to_16_iterator(
            const utf_8_to_16_iterator<I2, S2, ErrorHandler>& other);

    constexpr I begin() const { return it_.begin(); }

```

```

constexpr S end() const { return it_.end(); }

constexpr uint16_t operator*() const { return buf_[index_]; }

constexpr I base() const { return it_.base(); }

constexpr utf_8_to_16_iterator& operator++();
constexpr utf_8_to_16_iterator& operator--();

template<
    class I1, class S1,
    class I2, class S2,
    class ErrorHandler2>
friend constexpr bool operator==(
    const utf_8_to_16_iterator<I1, S1, ErrorHandler2>& lhs,
    const utf_8_to_16_iterator<I2, S2, ErrorHandler2>& rhs)
    requires requires { lhs.base() == rhs.base(); }
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

    friend constexpr bool operator==(utf_8_to_16_iterator lhs,
    utf_8_to_16_iterator rhs)
    { return lhs.base() == rhs.base() && lhs.index_ == rhs.index_; }

using base-type =                // exposition only
    iterator_interface<utf_8_to_16_iterator<I, S, ErrorHandler>,
        bidirectional_iterator_tag,
        uint16_t,
        uint16_t>;
using base-type::operator++;
using base-type::operator--;

private:
    utf_8_to_32_iterator<I, S> it_; // exposition only
    int index_;                     // exposition only
    array<uint16_t, 4> buf_;        // exposition only

    template<
        utf8_iter I2,
        sentinel_for<I2> S2,
        transcoding_error_handler ErrorHandler2>
    friend struct utf_8_to_16_iterator;
};

template<class I, class S, class ErrorHandler>
constexpr bool operator==(
    const utf_8_to_16_iterator<I, S, ErrorHandler>& lhs, Sentinel rhs)
    requires requires { lhs.base() == rhs; };
}

```

§ 4.6.2 Add out and insert transcoding iterators

```

namespace std::uc {
    template<class D, class I>
    struct trans-ins-iter { // exposition only
        using value_type = void;
    };
}

```

```

using difference_type = ptrdiff_t;
using pointer = void;
using reference = void;
using iterator_category = output_iterator_tag;

constexpr trans-ins-iter() {}
constexpr trans-ins-iter(I it) : it_(it) {}
constexpr D& operator*() { return derived(); }
constexpr D& operator++() { return derived(); }
constexpr D operator++(int) { return derived(); }
constexpr I base() const { return it_; }

protected:
    constexpr I& iter() { return it_; }

private:
    constexpr D& derived() { return static_cast<D&>(*this); }
    I it_;
};

// UTF-32 -> UTF-8

template<output_iterator<char> Iter>
struct utf_32_to_8_out_iterator
    : trans-ins-iter<utf_32_to_8_out_iterator<Iter>, Iter> {
    constexpr utf_32_to_8_out_iterator() {}
    constexpr explicit utf_32_to_8_out_iterator(Iter it);

    constexpr utf_32_to_8_out_iterator& operator=(uint32_t cp);
};

template<class Cont>
struct utf_32_to_8_insert_iterator
    : trans-ins-iter<utf_32_to_8_insert_iterator<Cont>,
        insert_iterator<Cont>> {
    constexpr utf_32_to_8_insert_iterator() {}
    constexpr utf_32_to_8_insert_iterator(Cont& c, typename Cont::iterator
        it);

    constexpr utf_32_to_8_insert_iterator& operator=(uint32_t cp);
};

template<class Cont>
struct utf_32_to_8_front_insert_iterator
    : trans-ins-iter<utf_32_to_8_front_insert_iterator<Cont>,
        front_insert_iterator<Cont>> {
    constexpr utf_32_to_8_front_insert_iterator() {}
    constexpr explicit utf_32_to_8_front_insert_iterator(Cont& c);

    constexpr utf_32_to_8_front_insert_iterator& operator=(uint32_t cp);
};

template<class Cont>
struct utf_32_to_8_back_insert_iterator
    : trans-ins-iter<utf_32_to_8_back_insert_iterator<Cont>,
        back_insert_iterator<Cont>> {

```

```

constexpr utf_32_to_8_back_insert_iterator() {}
constexpr explicit utf_32_to_8_back_insert_iterator(Cont& c);

constexpr utf_32_to_8_back_insert_iterator& operator=(uint32_t cp);
};

// UTF-8 -> UTF-32

template<output_iterator<uint32_t> Iter>
struct utf_8_to_32_out_iterator
    : trans-ins-iter<utf_8_to_32_out_iterator<Iter>, Iter> {
    constexpr utf_8_to_32_out_iterator() {}
    constexpr explicit utf_8_to_32_out_iterator(Iter it);

    constexpr utf_8_to_32_out_iterator& operator=(char cu);
};

template<class Cont>
struct utf_8_to_32_insert_iterator
    : trans-ins-iter<utf_8_to_32_insert_iterator<Cont>,
        insert_iterator<Cont>> {
    constexpr utf_8_to_32_insert_iterator() {}
    constexpr utf_8_to_32_insert_iterator(Cont& c, typename Cont::iterator
        it);

    constexpr utf_8_to_32_insert_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_8_to_32_front_insert_iterator
    : trans-ins-iter<utf_8_to_32_front_insert_iterator<Cont>,
        front_insert_iterator<Cont>> {
    constexpr utf_8_to_32_front_insert_iterator() {}
    constexpr explicit utf_8_to_32_front_insert_iterator(Cont& c);

    constexpr utf_8_to_32_front_insert_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_8_to_32_back_insert_iterator
    : trans-ins-iter<utf_8_to_32_back_insert_iterator<Cont>,
        back_insert_iterator<Cont>> {
    constexpr utf_8_to_32_back_insert_iterator() {}
    constexpr explicit utf_8_to_32_back_insert_iterator(Cont& c);

    constexpr utf_8_to_32_back_insert_iterator& operator=(uint16_t cu);
};

// UTF-32 -> UTF-16

template<output_iterator<uint16_t> Iter>
struct utf_32_to_16_out_iterator
    : trans-ins-iter<utf_32_to_16_out_iterator<Iter>, Iter> {
    constexpr utf_32_to_16_out_iterator() {}
    constexpr explicit utf_32_to_16_out_iterator(Iter it);

```

```

    constexpr utf_32_to_16_out_iterator& operator=(uint32_t cp);
};

template<class Cont>
struct utf_32_to_16_insert_iterator
    : trans-ins-iter<utf_32_to_16_insert_iterator<Cont>,
      insert_iterator<Cont>> {
    constexpr utf_32_to_16_insert_iterator() {}
    constexpr utf_32_to_16_insert_iterator(Cont& c, typename
      Cont::iterator it);

    constexpr utf_32_to_16_insert_iterator& operator=(uint32_t cp);
};

template<class Cont>
struct utf_32_to_16_front_insert_iterator
    : trans-ins-iter<utf_32_to_16_front_insert_iterator<Cont>,
      front_insert_iterator<Cont>> {
    constexpr utf_32_to_16_front_insert_iterator() {}
    constexpr explicit utf_32_to_16_front_insert_iterator(Cont& c);

    constexpr utf_32_to_16_front_insert_iterator& operator=(uint32_t cp);
};

template<class Cont>
struct utf_32_to_16_back_insert_iterator
    : trans-ins-iter<utf_32_to_16_back_insert_iterator<Cont>,
      back_insert_iterator<Cont>> {
    constexpr utf_32_to_16_back_insert_iterator() {}
    constexpr explicit utf_32_to_16_back_insert_iterator(Cont& c);

    constexpr utf_32_to_16_back_insert_iterator& operator=(uint32_t cp);
};

// UTF-16 -> UTF-32

template<output_iterator<uint32_t> Iter>
struct utf_16_to_32_out_iterator
    : trans-ins-iter<utf_16_to_32_out_iterator<Iter>, Iter> {
    constexpr utf_16_to_32_out_iterator() {}
    constexpr explicit utf_16_to_32_out_iterator(Iter it);

    constexpr utf_16_to_32_out_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_16_to_32_insert_iterator
    : trans-ins-iter<utf_16_to_32_insert_iterator<Cont>,
      insert_iterator<Cont>> {
    constexpr utf_16_to_32_insert_iterator() {}
    constexpr utf_16_to_32_insert_iterator(Cont& c, typename
      Cont::iterator it);

    constexpr utf_16_to_32_insert_iterator& operator=(uint16_t cu);
};

```

```

template<class Cont>
struct utf_16_to_32_front_insert_iterator
    :    trans-ins-iter<utf_16_to_32_front_insert_iterator<Cont>,
      front_insert_iterator<Cont>> {
    constexpr utf_16_to_32_front_insert_iterator() {}
    explicit constexpr utf_16_to_32_front_insert_iterator(Cont& c);

    constexpr utf_16_to_32_front_insert_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_16_to_32_back_insert_iterator
    :    trans-ins-iter<utf_16_to_32_back_insert_iterator<Cont>,
      back_insert_iterator<Cont>> {
    constexpr utf_16_to_32_back_insert_iterator() {}
    explicit constexpr utf_16_to_32_back_insert_iterator(Cont& c);

    constexpr utf_16_to_32_back_insert_iterator& operator=(uint16_t cu);
};

// UTF-16 -> UTF-8

template<output_iterator<char> Iter>
struct utf_16_to_8_out_iterator
    : trans-ins-iter<utf_16_to_8_out_iterator<Iter>, Iter> {
    constexpr utf_16_to_8_out_iterator() {}
    explicit constexpr utf_16_to_8_out_iterator(Iter it);

    constexpr utf_16_to_8_out_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_16_to_8_insert_iterator
    :    trans-ins-iter<utf_16_to_8_insert_iterator<Cont>,
      insert_iterator<Cont>> {
    constexpr utf_16_to_8_insert_iterator() {}
    constexpr utf_16_to_8_insert_iterator(Cont& c, typename Cont::iterator
      it);

    constexpr utf_16_to_8_insert_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_16_to_8_front_insert_iterator
    :    trans-ins-iter<utf_16_to_8_front_insert_iterator<Cont>,
      front_insert_iterator<Cont>> {
    constexpr utf_16_to_8_front_insert_iterator() {}
    explicit constexpr utf_16_to_8_front_insert_iterator(Cont& c);

    constexpr utf_16_to_8_front_insert_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_16_to_8_back_insert_iterator
    :    trans-ins-iter<utf_16_to_8_back_insert_iterator<Cont>,
      back_insert_iterator<Cont>> {

```

```

constexpr utf_16_to_8_back_insert_iterator() {}
explicit constexpr utf_16_to_8_back_insert_iterator(Cont& c);

constexpr utf_16_to_8_back_insert_iterator& operator=(uint16_t cu);
};

// UTF-8 -> UTF-16

template<output_iterator<uint16_t> Iter>
struct utf_8_to_16_out_iterator
    : trans-ins-iter<utf_8_to_16_out_iterator<Iter>, Iter> {
    constexpr utf_8_to_16_out_iterator() {}
    explicit constexpr utf_8_to_16_out_iterator(Iter it);

    constexpr utf_8_to_16_out_iterator& operator=(char cu);
};

template<class Cont>
struct utf_8_to_16_insert_iterator
    : trans-ins-iter<utf_8_to_16_insert_iterator<Cont>,
      insert_iterator<Cont>> {
    constexpr utf_8_to_16_insert_iterator() {}
    constexpr utf_8_to_16_insert_iterator(Cont& c, typename Cont::iterator
      it);

    constexpr utf_8_to_16_insert_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_8_to_16_front_insert_iterator
    : trans-ins-iter<utf_8_to_16_front_insert_iterator<Cont>,
      front_insert_iterator<Cont>> {
    constexpr utf_8_to_16_front_insert_iterator() {}
    explicit constexpr utf_8_to_16_front_insert_iterator(Cont& c);

    constexpr utf_8_to_16_front_insert_iterator& operator=(uint16_t cu);
};

template<class Cont>
struct utf_8_to_16_back_insert_iterator
    : trans-ins-iter<utf_8_to_16_back_insert_iterator<Cont>,
      back_insert_iterator<Cont>> {
    constexpr utf_8_to_16_back_insert_iterator() {}
    explicit constexpr utf_8_to_16_back_insert_iterator(Cont& c);

    constexpr utf_8_to_16_back_insert_iterator& operator=(uint16_t cu);
};
}

```

§ 4.6.3 Add factory functions for all the transcoding iterators

```

namespace std::uc {
    template<output_iterator<char> O>
        constexpr utf_32_to_8_out_iterator<O> utf_32_to_8_out(O it);

    template<output_iterator<uint32_t> O>

```

```

constexpr utf_8_to_32_out_iterator<0> utf_8_to_32_out(0 it);

template<output_iterator<uint16_t> 0>
constexpr utf_32_to_16_out_iterator<0> utf_32_to_16_out(0 it);

template<output_iterator<uint32_t> 0>
constexpr utf_16_to_32_out_iterator<0> utf_16_to_32_out(0 it);

template<output_iterator<char> 0>
constexpr utf_16_to_8_out_iterator<0> utf_16_to_8_out(0 it);

template<output_iterator<uint16_t> 0>
constexpr utf_8_to_16_out_iterator<0> utf_8_to_16_out(0 it);

template<bidirectional_iterator I, sentinel_for<I> S>
constexpr auto utf8_iterator(I first, I it, S last);

template<bidirectional_iterator I, sentinel_for<I> S>
constexpr auto utf16_iterator(I first, I it, S last);

template<bidirectional_iterator I, sentinel_for<I> S>
constexpr auto utf32_iterator(I first, I it, S last);

template<class Cont>
concept utf-container = // exposition only
    requires { typename Cont::value_type; } &&
    utf_code_unit<Cont::value_type>;

template<utf-container Cont>
constexpr auto from_utf8_inserter(Cont& c, typename Cont::iterator
    it);

template<utf-container Cont>
constexpr auto from_utf16_inserter(Cont& c, typename Cont::iterator
    it);

template<utf-container Cont>
constexpr auto from_utf32_inserter(Cont& c, typename Cont::iterator
    it);

template<utf-container Cont>
constexpr auto from_utf8_back_inserter(Cont& c);

template<utf-container Cont>
constexpr auto from_utf16_back_inserter(Cont& c);

template<utf-container Cont>
constexpr auto from_utf32_back_inserter(Cont& c);

template<utf-container Cont>
constexpr auto from_utf8_front_inserter(Cont& c);

template<utf-container Cont>
constexpr auto from_utf16_front_inserter(Cont& c);

template<utf-container Cont>

```



```
constexpr auto from_utf32_front_inserter(Cont& c);
}
```

§ 4.7 Add transcoding views

§ 4.7.1 Add the views proper

```
namespace std::uc {
    template<utf8_iter I, sentinel_for<I> S = I>
    struct utf8_view : view_interface<utf8_view<I, S>> {
        using iterator = I;
        using sentinel = S;

        constexpr utf8_view() {}
        constexpr utf8_view(iterator first, sentinel last);

        constexpr iterator begin() const;
        constexpr sentinel end() const;

        friend constexpr bool operator==(utf8_view lhs, utf8_view rhs)
            { return lhs.begin() == rhs.begin() && lhs.end() == rhs.end(); }

        template<class CharT, class Traits>
        friend basic_ostream<CharT, Traits>&
            operator<<(basic_ostream<CharT, Traits>& os, utf8_view v);

    private:
        using iterator_t = unspecified;           // exposition only
        using sentinel_t = unspecified;           // exposition only

        iterator_t first_;                        // exposition only
        [[no_unique_address]] sentinel_t last_;   // exposition only
    };

    template<utf16_iter I, sentinel_for<I> S = I>
    struct utf16_view : view_interface<utf16_view<I, S>> {
        using iterator = I;
        using sentinel = S;

        constexpr utf16_view() {}
        constexpr utf16_view(iterator first, sentinel last);

        constexpr iterator begin() const;
        constexpr sentinel end() const;

        friend constexpr bool operator==(utf16_view lhs, utf16_view rhs)
            { return lhs.begin() == rhs.begin() && lhs.end() == rhs.end(); }

        template<class CharT, class Traits>
        friend basic_ostream<CharT, Traits>&
            operator<<(basic_ostream<CharT, Traits>& os, utf16_view v);

    private:
        using iterator_t = unspecified;           // exposition only
        using sentinel_t = unspecified;           // exposition only
    };
}
```

```

    iterator_t first_; // exposition only
    [[no_unique_address]] sentinel_t last_; // exposition only
};

template<utf32_iter I, sentinel_for<I> S = I>
struct utf32_view : view_interface<utf32_view<I, S>> {
    using iterator = I;
    using sentinel = S;

    constexpr utf32_view() {}
    constexpr utf32_view(iterator first, sentinel last);

    constexpr iterator begin() const;
    constexpr sentinel end() const;
    friend constexpr bool operator==(utf32_view lhs, utf32_view rhs)
        { return lhs.begin() == rhs.begin() && lhs.end() == rhs.end(); }

    template<class CharT, class Traits>
        friend basic_ostream<CharT, Traits>&
            operator<<(basic_ostream<CharT, Traits>& os, utf32_view v);

private:
    using iterator_t = unspecified; // exposition only
    using sentinel_t = unspecified; // exposition only

    iterator_t first_; // exposition only
    [[no_unique_address]] sentinel_t last_; // exposition only
};

namespace std::ranges {
    template<class I, class S>
        inline constexpr bool enable_borrowed_range<uc::utf8_view<I, S>> =
            true;

    template<class I, class S>
        inline constexpr bool enable_borrowed_range<uc::utf16_view<I, S>> =
            true;

    template<class I, class S>
        inline constexpr bool enable_borrowed_range<uc::utf32_view<I, S>> =
            true;
}

```

§ 4.7.2 Add as_utfN()

Each `as_utfN()` factory function takes an iterator/sentinel pair, or a range-like (meaning an range or a null-terminated pointer), and returns a `utfN_view` that may do transcoding (if the inputs are not UTF-N) or may not do transcoding (if the inputs are UTF-N).

```

namespace std::uc {
    template<utf_iter I, sentinel_for<I> S>
        constexpr unspecified as_utf8(I first, S last);
}

```

```

template<utf_range_like R>
    constexpr unspecified as_utf8(R&& r);

template<utf_iter I, sentinel_for<I> S>
    constexpr unspecified as_utf16(I first, S last);

template<utf_range_like R>
    constexpr unspecified as_utf16(R&& r);

template<utf_iter I, sentinel_for<I> S>
    constexpr unspecified as_utf32(I first, S last);

template<utf_range_like R>
    constexpr unspecified as_utf32(R&& r);
}

```

§ 4.7.3 Add utfN_view specializations of formatter

These should be added to the list of “the debug-enabled string type specializations” in [format.formatter.spec]. This allows all three kinds of UTF views to be used in `std::format()` and `std::print()`. The intention is that each one will transcode to UTF-8 if the formatter’s `charT` is `char`, or to UTF-16 if the formatter’s `charT` is `wchar_t` – if transcoding is necessary at all.

```

template<class I, class S>
    struct formatter<uc::utf8_view<I, S>, charT>;
template<class I, class S>
    struct formatter<uc::utf16_view<I, S>, charT>;
template<class I, class S>
    struct formatter<uc::utf32_view<I, S>, charT>;

```

§ 4.8 Add a feature test macro

Add the feature test macro `__cpp_lib_unicode_transcoding`.

§ 4.9 Design notes

None of the proposed interfaces is subject to change in future versions of Unicode; each relates to the guaranteed-stable subset. Just sayin’.

None of the proposed interfaces allocates.

The proposed interfaces allow users to choose amongst multiple convenience-vs-compatibility-vs-performance tradeoffs. Explicitly, they are:

- If you need maximum performance, stick to the transcoding algorithms, and in particular use pointers for input and output.
- If you need compatibility with existing iterator-based algorithms (such as the standard algorithms), use the transcoding iterators.

- If you want streamability or the convenience of constructing ranges with a single `as_utfN()` function call, use the transcoding views.

All the transcoding iterators allow you access to the underlying iterator via `.base()`, following the convention of the iterator adaptors already in the standard.

The transcoding views are lazy, as you'd expect. They also compose with the standard view adaptors, so just transcoding at most 10 UTF-16 code units out of some UTF can be done with `std::uc::as_utf16(foo) | std::ranges::views::take(10)`.

Error handling is explicitly configurable in the transcoding iterators. This gives complete control to those who want to do something other than the default. The default, according to Unicode, is to produce a replacement character (`0xfffd`) in the output when broken UTF encoding is seen in the input. This is what all these interfaces do, unless you configure one of the iterators as mentioned above.

The production of replacement characters as error-handling strategy is good for memory compactness and safety. It allows use to store all our text as UTF-8 (or, less compactly, as UTF-16), and then process code points as transcoding views. If an error occurs, the transcoding views will simply produce a replacement character; there is no danger of UB.

Code units are just numbers. All of these interfaces treat integral types as code units of various sizes (at least the ones that are 8-, 16-, or 32-bit). Signedness is ignored.

A null-terminated pointer `p` to an 8-, 16-, or 32-bit string of code units is considered the implicit range `[p, null_sentinel)`. This makes user code much more natural; `as_utf16("foo")`, `as_utf16("foo"sv)`, and `as_utf16("foo"s)` are roughly equivalent (though the iterator type of the resulting view may differ).

Iterators are constructed from more than one underlying iterator. To do iteration in many text-handling contexts, you need to know the beginning and the end of the range you are iterating over, just to be able to do iteration correctly. Note that this is not a safety issue, but a correctness one. For example, say we have a string `s` of UTF-8 code units that we would like to iterate over to produce UTF-32 code points. If the last code unit in `s` is `0xe0`, we should expect two more code units to follow. They are not present, though, because `0xe0` is the last code unit. Now consider how you would implement `operator++()` for an iterator `iter` that transcodes from UTF-8 to UTF-32. If you advance far enough to get the next UTF-32 code point in each call to `operator++()`, you may run off the end of `s` when you find `0xe0` and try to read two more code units. Note that it does not matter that `iter` probably comes from a range with an end-iterator or sentinel as its mate; inside `iter`'s `operator++()` this is no help. `iter` must therefore have the end-iterator or sentinel as a data member. The same logic applies to the other end of the range if `iter` is bidirectional — it must also have the iterator to the start of the underlying range as a data member. This unfortunate reality comes up over and over in the proposed iterators, not just the ones that are UTF transcoding iterators. This is why iterators in this proposal (and the ones to come) usually consist of three underlying iterators.

§ 4.9.1 Iterator “unpacking”

A simple way to represent a transcoding view is as a pair of transcoding iterators. However, there is a problem with that approach, since a `utf32_view<utf_8_to_32_iterator<char const*>>` would be a range the size of 6 pointers. Worse yet, a `utf32_view<utf_8_to_16_iterator<utf_16_to_32_iterator<char const*>>>` would be the size of 18 pointers! Further, such a view would do a UTF-8 to UTF-16 to UTF-32 conversion, when it could have done a direct UTF-8 to UTF-32 conversion instead.

To solve these kinds of problems, `as_utfN()` unpacks the iterators it is given, so that only the bottom-most underlying pointer or iterator is stored:

```
std::string str = "some text";

auto to_16_first = std::uc::utf_8_to_16_iterator<std::string::iterator>(
    str.begin(), str.begin(), str.end());
auto to_16_last = std::uc::utf_8_to_16_iterator<std::string::iterator>(
    str.begin(), str.end(), str.end());

auto to_32_first = std::uc::utf_16_to_32_iterator<
    std::uc::utf_8_to_16_iterator<std::string::iterator>
>(to_16_first, to_16_first, to_16_last);
auto to_32_last = std::uc::utf_16_to_32_iterator<
    std::uc::utf_8_to_16_iterator<std::string::iterator>
>(to_16_first, to_16_last, to_16_last);

auto range = std::uc::as_utf8(to_32_first, to_32_last);

// Poof! The utf_16_to_32_iterators disappeared!
static_assert(std::is_same<decltype(range),
    std::uc::utf8_view<std::string::iterator>>::value, "");
```

Each of these views stores only the unpacked iterator and sentinel, so each view is typically the size of two pointers, and possibly smaller if a sentinel is used.

The same unpacking logic is used in `utfN_iterator()`, `from_utfN_inserter()`, the transcoding algorithms, and the normalization algorithms. This allows you to write `std::uc::as_utf32(first, last)` in a generic context, without caring whether `first` and `last` are iterators to a sequence of UTF-8, UTF-16, or UTF-32. You also do not need to care about whether `first` and `last` are raw pointers, some other kind of iterator, or transcoding iterators. For example, if `first` is a `utf_32_to_8_iterator`, the resulting view will use `first.base()` for its begin-iterator.

Sometimes, an interface might accept any UTF-N iterator, and then transcode internally to UTF-32:

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char> O>
    requires(utf8_code_unit<iter_value_t<I>> ||
             utf16_code_unit<iter_value_t<I>>)
transcode_result<I, O> transcode_to_utf32(I first, S last, O out);
```

For such interfaces, it can be difficult in the general case to form an iterator of type `I` to return to the user:

```
template<input_iterator I, sentinel_for<I> S, output_iterator<char> O>
    requires(utf8_code_unit<iter_value_t<I>> ||
             utf16_code_unit<iter_value_t<I>>)
transcode_result<I, O> transcode_to_utf32(I first, S last, O out) {
    // Get the input as UTF-32.
    auto r = uc::as_utf32(first, last);

    // Do transcoding.
    auto copy_result = ranges::copy(r, out);

    // Return an in_out_result.
    return result<I, O>{/* ??? */, copy_result.out};
}
```

What should we write for `/* ??? */`? That is, how do we get back from the UTF-32 iterator `r.begin()` to an `I` iterator? It's harder than it first seems; consider the case where

`I` is `std::uc::utf_16_to_32_iterator<std::uc::utf_8_to_16_iterator<std::string::iterator>>`.

The solution is for the unpacking algorithm to remember the structure of whatever iterator it unpacks, and then rebuild the structure when returning the result. To demonstrate, here is the implementation of `transcode_to_utf32` from Boost.Text:

```
template<std::input_iterator I, std::sentinel_for<I> S,
        std::output_iterator<uint32_t> O>
    requires(utf8_code_unit<std::iter_value_t<I>> ||
             utf16_code_unit<std::iter_value_t<I>>)
transcode_result<I, O> transcode_to_utf32(I first, S last, O out)
{
    auto const r = detail::unpack_iterator_and_sentinel(first, last);
    auto unpacked =
        detail::transcode_to_32<false>(r.tag_, r.f_, r.l_, -1, out);
    return {r.repack_(unpacked.in), unpacked.out};
}
```

If this all sounds way too complicated, it's not that bad at all. Here's the unpacking/repacking implementation from Boost.Text: [unpack.hpp](#).

§ 5 Implementation experience

All the interfaces proposed here have been implemented, and re-implemented, several times over the last 5 years or so. They are part of a proposed (but not yet accepted!) Boost library, [Boost.Text](#).

The library has hundreds of stars, though I’m not sure how many users that equates to. All of the interfaces proposed here are among the best-exercised in the library. There are comprehensive tests for all the proposed entities, and those entities are used as the foundation upon which all the other library entities are composed.

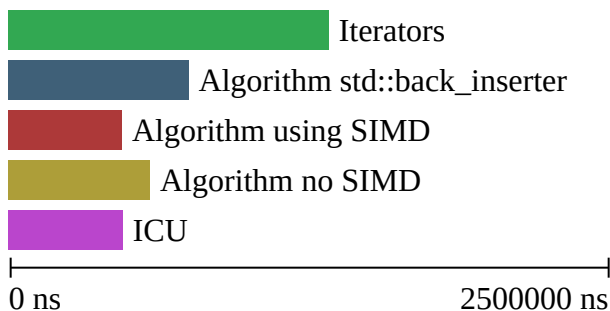
Though there are a lot of individual entities proposed here, at one time or another I have need each one of them, though maybe not in every UTF-N -> UTF-M permutation. Those transcoding permutations are there mostly for completeness. I have only ever needed UTF-8 <-> UTF->32 in any of my work that uses Unicode. Frequent Windows users will also need to convert to and from UTF-16 sometimes, because that is the UTF that the OS APIs use.

§ 6 Performance

The performance situation for UTF transcoding is complicated, and so bears some discussion. All the charts below were generated using Google Benchmark, built with GCC on Linux.

§ 6.1 UTF-8 to UTF-16

Here are the relative timings for UTF-8 to UTF-16 transcoding, using various methods (smaller is better). The input was around half a megabyte of text from Wikipedia. “Iterators” is using `std::copy` from `utf_8_to_16_iterator` to a pointer; “Algorithm `std::back_inserter`” is using `transcode_to_utf16()` in the SIMD code path, and outputting to a `std::back_insert_iterator`; “Algorithm using SIMD” is using `transcode_to_utf16()` from pointer to pointer in the SIMD-using code path; “Algorithm no SIMD” is using `transcode_to_utf16()` from pointer to pointer in the non-SIMD code path; and “ICU” is using `UnicodeString::fromUTF8()`.



The ICU performance is shown as something of a baseline, given the ubiquity of ICU’s use in Unicode-aware programs. Note that ICU does not have convenient APIs for doing transcoding to any format but UTF-16.

There are some take-always from this chart (and in fact all the other transcoding data):

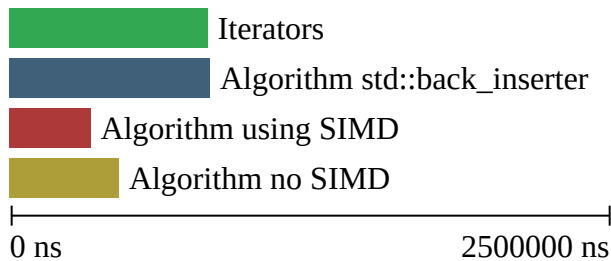
- The use of SIMD instructions is helpful, but not critical.
- The use of back-inserters is quite bad for performance.

- The transcoding iterators are terrible for performance.
- All the above only apply to transcode-only operations; more complicated operations that involve a transcoding step are often fairly insensitive to transcoding performance.
- The fastest API proposed is as fast as the equivalent ICU API.

A major reason for the performance differences is that the fastest algorithms are able to write out chunks of their results all in one go (up to 16 at once in the SIMD paths of the transcode algorithms). Needing to branch on each output code unit as in the “Iterators” and “Algorithm `std::back_inserter`” cases is much slower. One implication of this is that if you’re doing a lot of work with each code unit or code point produced, you’re probably doing a lot of branching in the work, and so the gains of using the high-performance methods above will be lost. Specifically, passing transcoding iterators to complicated Unicode algorithms like the Bidirectional Algorithm do not result in much (if any) performance loss.

§ 6.2 UTF-8 to UTF-32

These are relative timings for UTF-8 to UTF-32 transcoding. It is in the same scale as the chart above.



Again, you can see very similar relationships among the different transcoding methods, except that the iterator method is a lot faster.

Note that the SIMD algorithm is quite fast. It — and all the SIMD code — was originally developed by Bob Steagall, and presented at C++Now in 2018. Thanks, Bob!

§ 7 References

[P1629R1] JeanHeyd Meneide. 2020-03-02. Transcoding the world - Standard Text Encoding.

<https://wg21.link/p1629r1>